

Análisis Lógico: modelo_mlp.py

Desglose arquitectónico y funcional de la Red Neuronal para Scheduling

entrenar_mlp() Construcción y optimización del modelo

Esta función es el núcleo de aprendizaje. Su objetivo es tomar un conjunto de datos (features y labels) y ajustar los pesos de la red neuronal para que aprenda a relacionar el perfil de un empleado con la idoneidad para un turno.

Lógica de Ejecución

- 1 **Definición Arquitectónica:**
Instancia un modelo
`Sequential` de Keras. Crea
una estructura de embudo
(64 → 32 → 16 neuronas)
con activación `ReLU` para

Código Core

```
def entrenar_mlp(X, y, epochs=50, batch_size=32):  
    modelo = keras.Sequential([  
        layers.Input(shape=(X.shape[1],)),  
        layers.Dense(64, activation='relu'),  
        layers.Dense(32, activation='relu'),  
        layers.Dense(16, activation='relu'),
```

capturar relaciones no lineales complejas entre las características del turno y el empleado.

2 **Capa de Salida:** Utiliza una única neurona con activación `Sigmoid` para garantizar que la salida sea estrictamente una probabilidad (valor entre 0.0 y 1.0).

3 **Compilación:** Asigna `adam` como optimizador (eficiente para ajuste de pesos) y `binary\_crossentropy` como función de pérdida, la cual es estándar para clasificación binaria (asignar o no asignar).

```
        layers.Dense(1, activation='sigmoid')
    ])
    modelo.compile(
        optimizer='adam',
        loss='binary_crossentropy',
        metrics=['accuracy']
    )
    # ... división y .fit()
    return modelo, historial
```

4 Validación Cruzada
Simple: Divide el dataset estáticamente en 80% entrenamiento y 20% validación para monitorear el sobreajuste (overfitting).

5 Entrenamiento: Ejecuta el método `.fit()`, iterando ``epochs`` veces sobre los datos en lotes (``batch_size``), y evalúa métricas finales.

predecir_asignacion()

Inferencia y aplicación de reglas de negocio operativas

El modelo matemático por sí solo no garantiza una asignación legal (puede predecir que alguien trabaje 100 horas si las features son engañosas). Esta función actúa como el puente entre la IA probabilística y la realidad operativa mediante una estrategia *Greedy* post-predicción.

Lógica de Ejecución

1 Generación de Matriz de Posibilidades:

Crea el producto cartesiano de todos los empleados y todos los turnos. Extrae el vector de features para cada par exacto.

2 Inferencia Batch: Pasa toda la matriz ('X_pred') por el MLP entrenado en un solo lote para obtener las probabilidades de éxito de cada par.

3 Ordenamiento de Prioridad: Ordena la

Código Core

```
# 1. Obtener probabilidades
probabilidades = modelo.predict(X_pred).flatten()

# 2. Asignar probabilidad a cada par y ordenar
for i, p in enumerate(pares):
    p["probabilidad"] = float(probabilidades[i])
pares.sort(key=lambda p: p["probabilidad"], reverse=True)

# 3. Asignación Greedy con restricciones
for p in pares:
    if len(asignados) >= requeridos: continue
    if not esta_disponible(emp, turno): continue
    if not puede_trabajar_mas(...): continue
    if ya_trabaja_ese_dia(...): continue

    # Si pasa todo, se asigna
    asignaciones[turno_id].append(emp_id)
```

lista completa de pares de mayor a menor probabilidad. Esto asegura que el algoritmo priorice las combinaciones que la IA considera más "ideales".

4

Filtro Hard-

Constraints: Itera sobre la lista ordenada. Antes de asignar, pasa por validadores estrictos: ¿El turno está lleno? ¿El empleado está disponible? ¿Supera sus horas máximas? ¿Ya trabaja ese día? Si falla alguna, se descarta la asignación sin importar qué tan alta fuera la

probabilidad de la red neuronal.

5 Registro y

Exportación:

Construye el estado final de las asignaciones, calcula las horas consumidas y exporta la estructura a ``asignacion_mlp.json``.

`guardar_historial()` Persistencia de métricas de telemetría

Extrae las métricas del objeto ``History`` devuelto por Keras (loss, accuracy, val_loss, val_accuracy) al final del entrenamiento y las serializa en un formato estándar JSON. Su utilidad

radica en permitir la posterior graficación de las curvas de aprendizaje sin necesidad de retener el modelo en memoria RAM.

```
if __name__ == "__main__":
```

Orquestador de ejecución (Pipeline de datos)

Es el punto de entrada que ejecuta el pipeline secuencial y dependiente requerido para que el entorno funcione.

1. generar_dataset() → 2. entrenar_mlp() → 3. guardar_historial() → 4. predecir_asignacion()